DS	**Session**	Stack	**Question Information**	Level 1 Challenge 44

Problem Description:

Dr. Malar was booking a tour package of IRCTC from Chennai to Delhi for his family.

Two of the relatives was interested in joining to this tour.

these two persons are studying engineering in computing technology. only one tickets are remaining in the IRCTC portal.

So, Dr. Malar decided to book one ticket for out of those persons also along with his family members.

she wants to identify the one person out of these persons. he decided to conduct a technical task to identify the right person to travel.

the task was that, implement two stack operations in an array

Can you help them to complete the task?

Constraints

0<n<5 only five elements has to be practiced for this operation

first element pushed into stack1

second element pushed into stack2, likewise elements pushed into alternative stacks vice versa.

Function Description

- Create a data structure *twoStacks* that represents two stacks.
- Implementation of *twoStacks* should use only one array, i.e., both stacks should use the same array for storing elements.
- Following functions must be supported by *twoStacks*.
  push1(int x) –> pushes x to first stack
  push2(int x) –> pushes x to second stack
  pop1() –> pops an element from first stack and return the popped element

pop2() –> pops an element from second stack and return the popped element
Implementation of *twoStack* should be space efficient.

Input Format

Single line represents only braces (both curly and square)

Output Format

If the given input balanced then print as Balanced (or) Not Balanced

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
2
3
4
5
```

EXPECTED OUTPUT

```
Popped element from stack1 is:5
Popped element from stack2 is:4
```

Test Case 2

INPUT (STDIN)

```
5
4
5
3
4
```

EXPECTED OUTPUT

```
Popped element from stack1 is:4
Popped element from stack2 is:3
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void push1(int x)
```

Test Case 2

KEYWORD

```
void push2(int x)
```

Test Case 3

KEYWORD

```
int pop1()
```

Test Case 4

KEYWORD

```
int pop2()
```

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
1	380	81